

پایپلاین Multi-Agent LLM برای گزارش‌گیری بالینی هوشمند

۱. مرور عملکرد تسک ۳

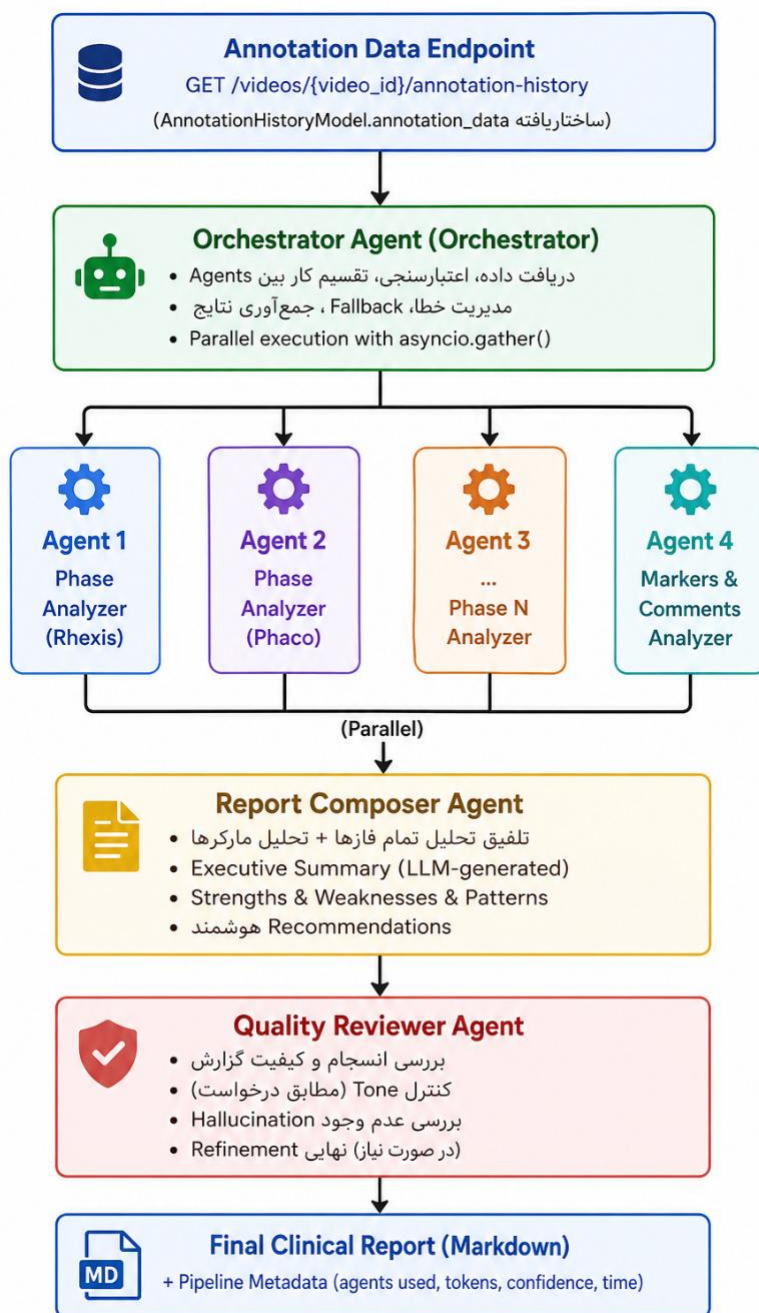
نکات مثبت

جنبه	تحلیل
معماری لایه‌ای	جدا کردن پوشه‌های Domain / Application / Infrastructure ایده‌آل برای extensibility
پایپلاین ناهمگام	۴ مرحله مستقل (phase_summary, score_analysis,) با ARQ worker (comment_timeline, contradictions)
یکپارچه‌سازی LLM	اتصال به Gemini 2.5 Flash از طریق REST مستقیم (بدون وابستگی به SDK)
مهندسی پرامپت	System Instruction با قوانین سخت بالینی (عدم جعل داده، عدم تجویز) + خروجی Markdown ساختاریافته
Rule Engine مستقل	تعریف تناقض‌ها در YAML و موتور ارزیابی مجزا، انعطاف‌پذیر بدون نیاز به تغییر کد
امنیت	HMAC-SHA256 برای sanitization, webhook, نام فایل، rate limiting, CORS
پشتیبان دو زبانه	locale-aware (EN/FA) در تمام خروجی‌ها
کیفیت کد	type hints, docstrings, error handling لایه‌بندی شده

نقاط قابل بهبود (رهنمود برای تسک ۵)

جنبه	وضعیت در تسک ۳	انتظار در تسک ۵
معماری Agent	تک LLM با یک پرامپت monolithic	چند Agent تخصصی با پرامپت مجزا
جداسازی پرامپت‌ها	System Instruction در کد (hardcoded)	فایل‌های txt. مجزا در پوشه prompts
تحلیل Markers	فقط در Rule Engine	Agent مجزا برای تحلیل معنایی مارکرها
Quality Review	ندارد	Agent مجزا برای بازبینی کیفیت
Fallback Rule-Based	در دسترس نبودن LLM = خطا	Fallback به report_service.py
موازی‌سازی Agents	ترتیبی (sequential stages)	پردازش موازی Phase ها با asyncio
Confidence Score	ندارد	امتیاز اطمینان برای هر بخش
Token Management	MaxOutputTokens ثابت	شمارش توکن + chunking در فازهای زیاد
کش (Caching)	ندارد	کش پاسخ‌های تکراری (همان phase + scores)
Provider Agnostic	فقط Gemini	قابل تعویض بین OpenAI / Anthropic / Ollama
تست سناریوهای LLM	تست integration محدود	۴ سناریو اجباری + تست Fallback

۳. معماری هدف Multi-Agent Pipeline :



۴. داده‌های ورودی (از پروژه SurgiNote)

۴.۱. منبع داده اصلی

اندپوینت واقعی پروژه (که در تسک ۳ هم استفاده شده):

GET /videos/{video_id}/annotation-history

خروجی شامل annotation_data با ساختار زیر است (بر اساس AnnotationHistoryModel در سمت کلاینت سرچینوت):

```
{
  "phases": [
    {
      "id": "phase_3",
      "order": 3,
      "name": "Rhexis",
      "shortName": "Rhexis",
      "description": "Continuous curvilinear capsulorhexis",
      "color": "#8B5CF6",
      "startTime": 30.5,
      "endTime": 125.0,
      "phacoMethod": null,
      "source": "manual"
    }
  ],
  "metrics": {
    "phase_3": [
      {"id": "rhexis_1", "name": "Microscope use & visualization", "maxScore":
5}
```

```
    ]
  },
  "scores": {
    "phase_3": {
      "rhexis_1": 4.5,
      "rhexis_2": 3.0
    }
  },
  "markers": [
    {
      "id": "marker_1",
      "type": "comment",
      "timestamp": 45.0,
      "label": "Good technique",
      "text": "Smooth continuous tear",
      "color": "#0ea5e9",
      "createdAt": "2026-01-15T10:31:00Z"
    }
  ],
  "version": 2,
  "savedAt": "2026-01-15T10:35:00Z"
}
```

۴.۲. موارد مورد نیاز دسترسی در پروژه (در پوشه کد قرار گرفته)

مسیر فایل	مؤلفه
AnnotationHistoryModel.js	مدل دامین (کلاينت)
annotationApi.js	API کلاينت
report_service.py	سرویس Rule-Based
surginote-report-pipeline-main/(تسک ۳)	پایپلاین قبلی

۵. معماری Agent ها (مشروح)

۵.۱. Agent 1: Data Extractor & Normalizer

نقش: داده خام API را به فرمت استاندارد برای Agents دیگر تبدیل می کند.

ورودی: خروجی JSON از GET /videos/{id}/annotation-history

خروجی: دیکشنری نرمالایز شده

با normalized_phases[], normalized_markers[], metadata{ }

System Prompt کلیدی:

You are a surgical annotation data normalizer. Your ONLY job is to parse, validate, and standardize the input JSON structure. Do NOT analyze or interpret the clinical data. Ensure all timestamps are in seconds, all IDs are strings, and all nested objects follow the canonical schema. Return ONLY valid JSON. If data is malformed, return {"error": "...", "normalized": null}.

فایل پرامپت :

prompts/data_extractor.txt

۵.۲. Agent 2: Phase Analyzer (چند نمونه موازی)

نقش: تحلیل یک فاز جراحی و تولید Narrative بالینی + شناسایی نقاط قوت/ضعف.

ورودی: یک فاز + متریک‌های آن + امتیازات (آبجکت تکی)

خروجی: تحلیل JSON شامل:

- clinical_narrative (۲-۴ جمله)
- strengths[] و weaknesses[]
- recommendations[]
- key_insights[]
- confidence_score (۰-۱)

System Prompt کلیدی:

You are a board-certified ophthalmology simulator specializing in cataract surgery evaluation. Analyze the given surgical phase using ONLY the provided metrics and scores. Never invent data. Use precise anatomical terminology. Your narrative must flow naturally without mentioning exact scores. Consider the tone level (0=Very Critical to 4=Very Encouraging) when phrasing feedback. Each phase has specific clinical meaning — e.g., Rhexis is about capsulorhexis integrity, Phaco is about chamber stability and nuclear management.

فایل پرامپت: prompts/phase_analysis.txt

۵.۳. Agent 3: Markers & Comments Analyzer

نقش: تحلیل معنایی مارک‌های زمانی و کامنت‌ها.

ورودی: آرایه‌ای از markers[] مرتب‌شده زمانی

خروجی:

- commentary_insights[], بینش‌های استخراج شده از متن کامنت‌ها
- emotion_trend, روند مثبت/منفی/خنثی کامنت‌ها
- critical_events[], رویدادهای مهم (warning/error markers)
- alignment_with_scores, تطابق کامنت‌ها با امتیازات

System Prompt کلیدی:

You are a clinical annotator analyzer. Review the timestamped markers and comments from a cataract surgery video. Extract meaningful patterns: Are comments mostly positive or corrective? Do they align with the numerical scores? Identify any discrepancies (e.g., high score but negative comment, or low score but no comment). Group related comments. Never speculate beyond the provided text.

فایل پرامپت: prompts/markers_analysis.txt

۵.۴. Agent 4: Report Composer

نقش: ترکیب خروجی تمام Phase Analyzer ها + Markers Analyzer در یک گزارش منسجم.

ورودی: خروجی تمام Agents دیگر (List[PhaseAnalysis] + MarkersAnalysis)

خروجی: گزارش Markdown کامل شامل Executive Summary, Overall Assessment, Recommendations

System Prompt کلیدی:

You are a senior surgical report composer. Synthesize the provided phase analyses and marker insights into a coherent, professional clinical report. Follow the exact Markdown structure specified. The Executive Summary must be a holistic assessment — not just averaging scores, but identifying patterns across phases.

Strengths & Weaknesses should be prioritized by clinical significance, not just score. Recommendations must be actionable and specific.

فایل پرامپت: prompts/report_composition.txt

۵.۵. Agent 5: Quality Reviewer

نقش: بازبینی گزارش نهایی قبل از تحویل.

ورودی: گزارش Markdown تولید شده توسط Composer

خروجی: گزارش بازبینی شامل:

- issues_found[] مشکلات (tone mismatch, hallucination risk, missing sections)
- suggested_fixes[] اصلاحات پیشنهادی
- overall_quality_score (۰-۱)
- approved: boolean، تأیید نهایی

System Prompt کلیدی:

You are a quality assurance reviewer for AI-generated clinical reports. Review the report for: (1) adherence to the requested tone, (2) clinical accuracy of terminology, (3) absence of hallucinations or invented data, (4) proper structure and completeness, (5) consistency between narrative and scores. Flag any issues with specific line references. If approved=False, provide specific revision instructions.

فایل پرامپت: prompts/quality_review.txt

۶. ساختار فایل‌ها

ساختار مازول Report

پوشه report/ شامل اجزای اصلی تولید گزارش و زیرسیستم جدید مبتنی بر LLM است.

۱. فایل‌های اصلی ماژول **Report**

- `__init__.py`
- `routes.py`
- جهت افزودن Endpoint جدید:
 - `/reports/generate-ai`
- `report_schemas.py`
- جهت افزودن مدل‌های:
 - `AISettings`
 - `AIReportResponse`
- `report_service.py`
- سرویس گزارش‌ساز Rule-Based که در سناریوهای Fallback مورد استفاده قرار می‌گیرد.
- `clinical_phrases.py`
- شامل عبارات و اصطلاحات استاندارد مورد استفاده در گزارش‌های بالینی.

۲. زیرسیستم جدید **AI Pipeline**

پوشه `ai_pipeline/` به منظور پیاده‌سازی معماری **Multi-Agent** مبتنی بر مدل‌های زبانی ایجاد شده است.

۲.۱. پی‌کربندی و ارتباط با مدل‌های زبانی

config.py

- مدیریت تنظیمات `Provider`
- انتخاب `Model`
- تنظیم `Temperature`
- مدیریت `API Keys`

- دریافت تنظیمات از طریق Environment Variables

llm_client.py

- پیاده‌سازی لایه انتزاعی ارتباط با مدل‌های زبانی
- ارائه کلاس LLMClient
- متد اصلی:
- chat(system_prompt, user_msg) -> str
- پشتیبانی از:
- OpenAI
- Anthropic
- Ollama
- Gemini

۲.۲. لایه Agent ها

base_agent.py

- تعریف کلاس پایه انتزاعی (BaseAgent)
- اجزای اصلی:
- system_prompt
- process(input) -> output
- پوشه agents/ شامل Agent های تخصصی زیر است:
- data_extractor.py
- Agent 1
- استخراج و آماده‌سازی داده‌ها
- phase_analyzer.py

Agent 2 ○

○ تحلیل فازهای جراحی

• markers_analyzer.py

Agent 3 ○

○ تحلیل Marker ها و کامنت‌ها

• report_composer.py

Agent 4 ○

○ ترکیب نتایج و تولید گزارش نهایی

• quality_reviewer.py

Agent 5 ○

○ بازبینی کیفیت و اصلاح خروجی نهایی

۲.۳. هماهنگ‌کننده مرکزی Pipeline

orchestrator.py

شامل کلاس PipelineOrchestrator با مسئولیت‌های زیر:

• مدیریت ترتیب اجرای Agent ها

• اجرای موازی تحلیل‌ها با استفاده از asyncio

• مدیریت خطاها و Fallback به سرویس Rule-Based

• مدیریت و پایش مصرف Token

• مدیریت Caching و استفاده مجدد از نتایج قبلی

۲.۴. مدیریت Prompt ها

پوشه prompts/ شامل تمامی Prompt های مورد استفاده در Agent ها است.

فایل‌ها:

- data_extractor.txt
- phase_analysis.txt
- markers_analysis.txt
- report_composition.txt
- quality_review.txt

این ساختار باعث جداسازی کامل Prompt ها از کد برنامه و افزایش قابلیت نگهداری سیستم می شود.

۲.۵. مدیریت Cache

cache.py

مسئول ذخیره سازی پاسخ های تولید شده توسط LLM .

فرمول تولید Cache Key :

`hash(phase_id + scores + tone + model)`

۲.۶. مدیریت مصرف Token

token_manager.py

مسئول:

- شمارش Token های ورودی و خروجی
- تقسیم داده ها (Chunking) در گزارش های بزرگ
- هشدار هنگام نزدیک شدن به محدودیت Token مدل

۲.۷. ابزارهای کمکی

utils.py

شامل توابع عمومی مورد نیاز Pipeline :

- parse_json_from_llm

- مدیریت JSON های ناقص یا نامعتبر تولیدشده توسط مدل

- retry_with_backoff

- اجرای مجدد درخواست ها با سیاست Backoff

- validate_llm_output

- اعتبارسنجی ساختار خروجی مدل

۳. فایل های مستندات و نمونه خروجی

sample_report_ai.md

- نمونه گزارش تولیدشده توسط AI Pipeline

TASK5_REPORT

- مستندات فنی Task 5

- توضیح معماری

- تصمیمات طراحی

- ملاحظات فنی و پیاده سازی

۷. جزئیات فنی پیاده سازی

۷.۱. کلاس پایه Agent

...

```
from abc import ABC, abstractmethod
from typing import Dict, Any, Optional
from pathlib import Path
```

```
class BaseAgent(ABC):
```

```
    """
```

```
    Abstract base class for all AI agents.
```

Each agent:

1. Loads its system prompt from a .txt file
2. Processes input data through an LLM call
3. Returns structured output (usually JSON)

Key design decisions:

- Prompts are in separate files, NOT in code
- Each agent has ONE responsibility
- Output format is enforced via prompt + post-processing

```
    """
```

```
def __init__(
    self,
    agent_name: str,
    prompt_path: str,
    llm_client: Any, # LLMClient instance
```

```

        temperature: float = 0.3,
        max_retries: int = 2,
    ):
        self.agent_name = agent_name
        self.llm_client = llm_client
        self.temperature = temperature
        self.max_retries = max_retries
        self.system_prompt = self._load_prompt(prompt_path)

    def _load_prompt(self, path: str) -> str:
        """Load system prompt from file."""
        full_path = Path(__file__).parent / "prompts" / path
        return full_path.read_text(encoding="utf-8")

    def _call_llm(self, user_message: str) -> str:
        """Call LLM with retry logic."""
        for attempt in range(self.max_retries):
            try:
                return self.llm_client.chat(
                    system_prompt=self.system_prompt,
                    user_message=user_message,
                    temperature=self.temperature,
                )
            except Exception as e:
                if attempt == self.max_retries - 1:

```



```
        raise
    continue
    return ""
```

```
@abstractmethod
def process(self, input_data: Dict[str, Any]) -> Dict[str, Any]:
    """Main processing method."""
    Pass
...

```

7.2. Orchestrator

```
...

import asyncio
from typing import Dict, Any, List
from datetime import datetime
```

```
class PipelineOrchestrator:
    """
    Orchestrates the multi-agent pipeline.
```

Flow:

1. Receive raw annotation data
2. DataExtractor → normalized format
3. PhaseAnalyzers (parallel) → per-phase analyses
4. MarkersAnalyzer → marker insights

5. ReportComposer → final report

6. QualityReviewer (optional) → review & refine

Fallback: If LLM fails entirely, use Task 4's rule-based report_service.py

"""

```
def __init__(self, config: Dict, llm_client: Any):
```

```
    self.config = config
```

```
    self.llm_client = llm_client
```

```
    self._init_agents()
```

```
def _init_agents(self):
```

```
    """Initialize all agents with their prompts and config."""
```

```
    ...
```

```
async def run(
```

```
    self,
```

```
    phases: List[Dict],
```

```
    metrics: Dict,
```

```
    scores: Dict,
```

```
    markers: List[Dict],
```

```
    settings: Dict,
```

```
) -> Dict:
```

```
    """
```

```
    Main entry point.
```

Returns:

```
{
    "content": str (Markdown report),
    "metadata": {
        "agents_used": [...],
        "tokens_used": int,
        "generation_time": float,
        "confidence_score": float,
        "fallback_used": bool,
        "quality_score": float (optional)
    }
}
"""

try:
    # 1. Normalize data
    normalized = await self.data_extractor.process({
        "phases": phases,
        "metrics": metrics,
        "scores": scores,
        "markers": markers,
    })

    # 2. Analyze phases in parallel (Map step)
    phase_tasks = [
```

```
self.phase_analyzer.process({
    "phase": p,
    "metrics": normalized["metrics"].get(p["id"], []),
    "scores": normalized["scores"].get(p["id"], {}),
    "tone": settings.get("tone", 2),
})
for p in normalized["phases"]
]
phase_analyses = await asyncio.gather(*phase_tasks)

# 3. Analyze markers
markers_analysis = await self.markers_analyzer.process({
    "markers": normalized["markers"],
})

# 4. Compose report (Reduce step)
report = await self.report_composer.process({
    "phase_analyses": phase_analyses,
    "markers_analysis": markers_analysis,
    "settings": settings,
    "metadata": normalized["metadata"],
})

# 5. Quality review (optional)
if self.config.get("enable_review", False):
```

```

        review = await self.quality_reviewer.process({
            "report": report["content"],
            "settings": settings,
        })
        if not review.get("approved", True):
            # Refinement logic here
            pass

    return report

except Exception as e:
    # Fallback to rule-based (Task 4)
    return self._fallback_to_rule_based(
        phases, metrics, scores, settings, error=str(e)
    )

def _fallback_to_rule_based(self, phases, metrics, scores, settings, error):
    """Use Task 4's report_service as fallback."""
    from ..report_service import generate_clinical_report
    content = generate_clinical_report(phases, metrics, scores, settings)
    return {
        "content": content,
        "metadata": {
            "fallback_used": True,
            "fallback_reason": error,
        }
    }

```

```
    }  
    }  
    ...
```

۷.۳. کلاينت LLM يکپارچه

```
class LLMClient:
```

```
    """
```

Unified LLM client supporting multiple providers.

Usage:

```
    client = LLMClient(provider="openai", model="gpt-4", api_key="...")
```

```
    response = client.chat(system_prompt="...", user_message="...")
```

Supported providers: openai, anthropic, ollama, gemini

Fallback: If primary provider fails, try secondary if configured

```
    """
```

```
def __init__(self, provider: str = "openai", **kwargs):
```

```
    self.provider = provider
```

```
    self.model = kwargs.get("model", "gpt-4")
```

```
    self.api_key = kwargs.get("api_key", "")
```

```
    self.temperature = kwargs.get("temperature", 0.3)
```

```
    self.max_tokens = kwargs.get("max_tokens", 2048)
```

```
    self.timeout = kwargs.get("timeout", 60)
```

```
    self._init_provider()
```



```
import json
from functools import lru_cache
```

```
class LLMCache:
```

```
    """
```

Cache for LLM responses to avoid redundant API calls.

Cache key = hash(agent_name + input_data_json + model + temperature)

TTL: configurable (default: 1 hour)

Why cache?

- Same phase + same scores → same analysis expected
- Reduces API costs by ~40-60% in repeated evaluations
- Faster response for common patterns

```
    """
```

```
    def __init__(self, ttl_seconds: int = 3600):
```

```
        self._cache = {}
```

```
        self.ttl = ttl_seconds
```

```
    def _make_key(self, agent_name: str, input_data: Dict, model: str, temperature: float) -> str:
```

```
        raw = f"{agent_name}:{json.dumps(input_data, sort_keys=True)}:{model}:{temperature}"
```

```
        return hashlib.sha256(raw.encode()).hexdigest()
```



```

def get(self, agent_name: str, input_data: Dict, model: str, temperature: float):
    key = self._make_key(agent_name, input_data, model, temperature)
    entry = self._cache.get(key)
    if entry and (datetime.now() - entry["timestamp"]).seconds < self.ttl:
        return entry["value"]
    return None

```

```

def set(self, agent_name: str, input_data: Dict, model: str, temperature: float,
value: str):
    key = self._make_key(agent_name, input_data, model, temperature)
    self._cache[key] = {"value": value, "timestamp": datetime.now()}
...

```

٨. API Endpoint

٨.١. Route جديد

```

...

@router.post(
    "/reports/generate-ai",
    response_model=GenerateReportResponse,
    summary="Generate a clinical report using AI-Agent pipeline",
)
async def generate_ai_report(request: GenerateReportRequest):
    """
    Multi-agent AI pipeline that:

```

1. Normalizes annotation data
2. Analyzes each phase with a specialized LLM agent (parallel)
3. Analyzes markers and comments for insights
4. Composes a comprehensive clinical report
5. (Optional) Reviews quality before returning

Falls back to rule-based service if LLM unavailable.

```
""  
  
...  
...
```

۸.۲. درخواست نمونه

```
{  
  "phases": [...],  
  "metrics": {...},  
  "scores": {...},  
  "markers": [  
    {"id": "m1", "type": "comment", "timestamp": 45.0, "label": "...", "text":  
"..."},  
    {"id": "m2", "type": "warning", "timestamp": 85.0, "label": "...", "text": "..."}  
  ],  
  "settings": {  
    "emphasis": ["technical", "safety"],  
    "tone": 2,  
    "ai_config": {  
      "provider": "openai",
```

```
"model": "gpt-4",
"temperature": 0.3,
"enable_review": true,
"enable_cache": true,
"parallel_phases": true
}
}
}
```

۸.۳. پاسخ نمونه

```
{
"content": "# Surgical Procedure Evaluation Report\n\n...",
"generatedAt": "2026-06-14T10:00:00Z",
"metadata": {
"pipeline": "ai-agent-v2",
"agents_used": [
"data_extractor",
"phase_analyzer(3 parallel instances)",
"markers_analyzer",
"report_composer",
"quality_reviewer"
],
"model": "gpt-4",
"tokens_used": {
"input": 4820,
"output": 1532,
```

```

    "total": 6352
  },
  "generation_time_seconds": 8.7,
  "confidence_score": 0.91,
  "fallback_used": false,
  "cache_hits": 2,
  "quality_score": 0.95
}
}

```

۹. سناریوهای تست اجباری

سناریو ۱: داده کامل (۳ فاز + ۴ مارکر **Tone** + معمولی)

فاز	مارکرها	امتیازات	متریک‌ها
Rhexis	۲ کامنت مثبت	۴.۵, ۳.۰, ۴.۰, ۲.۵, ۴.۵, ۴.۰	۶ متریک
Phacoemulsification	comment\ + warning\	۴.۵, ۳.۰, ۴.۰, ۵.۰, ۴.۵, ۴.۰	۷ متریک
I&A	—	۴.۵, ۲.۵, ۴.۰	۳ متریک

Tone : 2 (Balanced)

خروجی مورد انتظار : گزارش کامل با ۵ بخش، narrative روان، نقاط قوت/ضعف دقیق

سناریو ۲: داده حداقلی (۱ فاز، بدون مارکر)

Tone : 4 (Very Encouraging)

خروجی مورد انتظار : گزارش کوتاه ولی حرفه‌ای، لحن تشویق کننده

سناریو ۳: **Fallback (LLM)** در دسترس نیست

- قطع کردن API key یا شبکه
- خروجی مورد انتظار: گزارش Rule-Based با `metadata.fallback_used = true`

سناریو ۴ Tone Extremes :

داده مشابه سناریو ۱ با:

- 0 Tone (Very Critical) باید narrative تند و نقادانه باشد
- 4 Tone (Very Encouraging) باید narrative بسیار تشویق کننده باشد
- خروجی مورد انتظار: تفاوت محسوس در لحن بدون تغییر در محتوای فنی

۱۰. درس گرفته شده از تسک ۳ (آنچه باید تکرار شود)

۱. معماری تمیز Domain/Application/Infrastructure جداسازی شود
۲. Security first — API key در env — Sanitize ورودی، Rate limiting
۳. Error handling لایه بندی شده — هر Agent خطاهای خود را مدیریت کند
۴. Locale awareness — قابلیت گسترش به فارسی در آینده
۵. Logging ساختاریافته — هر Agent لاگ مجزا داشته باشد
- ۶.

۱۱. درس گرفته شده از تسک ۳ (آنچه باید اصلاح شود)

۱. پرامپت monolithic نه، پرامپت های تخصصی برای هر Agent
۲. LLM وابسته به یک provider نه، کلاینت انتزاعی با قابلیت تعویض
۳. بدون Fallback نه، Fallback اجباری به Rule-Based
۴. بدون Cache نه، کش هوشمند برای کاهش هزینه و افزایش سرعت
۵. بدون Confidence Score نه، امتیاز اطمینان برای هر بخش

۱۲. موارد تحویلی

فایل	توضیح
ai_pipeline/__init__.py	فایل خالی
ai_pipeline/config.py	تنظیمات provider, model, temperature
ai_pipeline/llm_client.py	کلاینت یکپارچه LLM
ai_pipeline/base_agent.py	کلاس پایه Abstract
ai_pipeline/agents/data_extractor.py	Agent نرمالایز داده
ai_pipeline/agents/phase_analyzer.py	Agent تحلیل فاز
ai_pipeline/agents/markers_analyzer.py	Agent تحلیل مارکر
ai_pipeline/agents/report_composer.py	Agent ترکیب گزارش
ai_pipeline/agents/quality_reviewer.py	Agent بازبینی کیفیت
ai_pipeline/orchestrator.py	هماهنگ کننده اصلی
ai_pipeline/prompts/*.txt	حداقل ۴ فایل پرامپت
ai_pipeline/cache.py	کش پاسخها
ai_pipeline/token_manager.py	مدیریت توکن
ai_pipeline/utils.py	توابع کمکی
routes.py	اضافه کردن /reports/generate-ai
report_schemas.py	اضافه کردن AISettings, AIReportResponse
sample_report_ai.md	خروجی نمونه با metadata کامل
TASK5_REPORT.md	گزارش فنی از معماری و تصمیمات

۱۳. راهنمای شروع سریع

مطالعه خروجی تسک ۳ (برای درس گرفتن)

`surginote-report-pipeline-main`

معماری و `narrative.py` و `report_pipeline.py` را بخوانید

کد `report_service.py` را مرور کنید برای Fallback

ساختار `annotation data` را بررسی کنید

پیاده‌سازی را شروع کنید

مرحله ۱: `config.py` + `llm_client.py`

مرحله ۲: `base_agent.py` + `prompts`

مرحله ۳: `phase_analyzer.py` (یک Agent کامل)

مرحله ۴: `orchestrator.py` (اتصال همه چیز)

مرحله ۵: `routes.py` + تست

مرحله ۶: `cache` + `token_manager` + `optimization`

۱۴. نکات نهایی

۱. کیفیت پرامپت <<< همه چیز. وقت کافی برای طراحی و تست پرامپت‌ها بگذارید.
۲. هر Agent یک کار انجام می‌دهد. اگر Agentی دارد دو کار متفاوت می‌کند، آن را بشکنید.
۳. **Parallel processing** اجباری است. فازها وابستگی به هم ندارند، باید موازی اجرا شوند.
۴. **Fallback** اجباری است. اگر LLM در دسترس نباشد، سرویس نباید از کار بیفتد.
۵. خروجی LLM را **validate** کنید. ممکن است JSON نامعتبر تولید کند.
۶. **Token consumption** را اندازه بگیرید. در metadata برگردانید.
۷. زبان خروجی: انگلیسی (نسخه فعلی). قابلیت گسترش به فارسی در `config`.

موفق باشید